

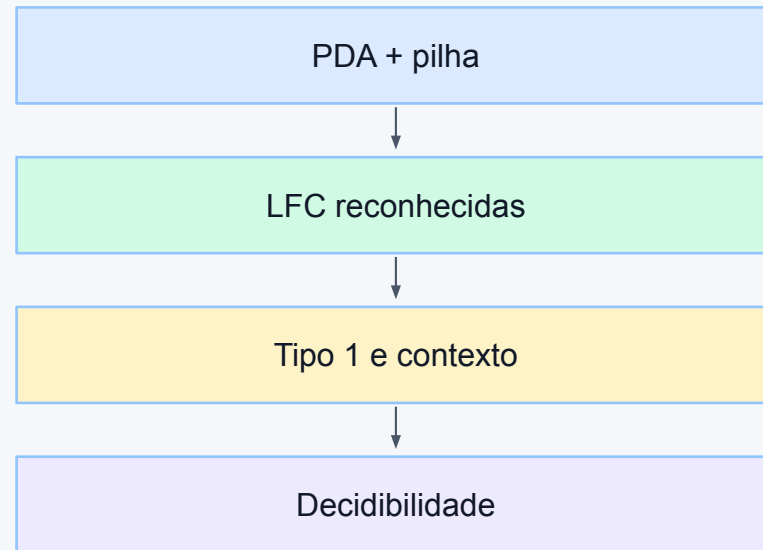
Autômatos com pilha e decidibilidade

LFC, pilha, gramáticas sensíveis ao contexto e limites da computação

Nesta aula avançamos além dos autômatos finitos. Veremos por que algumas linguagens precisam de memória em pilha, como autômatos com pilha reconhecem linguagens livres de contexto e como a hierarquia chega a gramáticas sensíveis ao contexto, linguagens recursivas e questões de decidibilidade.

[Link útil: JFLAP oficial](#)

Roteiro visual da aula

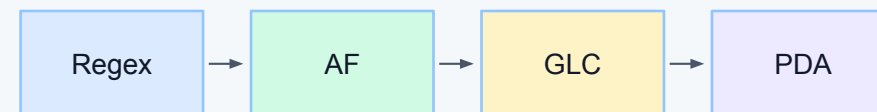


Continuidade da disciplina

Do regular ao livre de contexto

Nas aulas anteriores, linguagens regulares foram descritas por expressões regulares, gramáticas regulares e autômatos finitos. Agora, o foco muda para estruturas que exigem memória, como parênteses balanceados, blocos aninhados e expressões sintáticas. Essa transição aproxima a teoria da análise sintática em compiladores.

Da aula anterior à atual



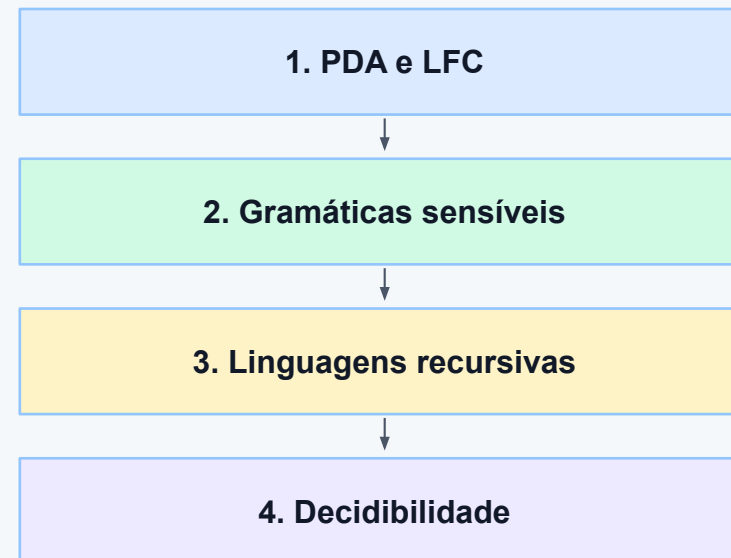
o parser precisa de memória

Mapa conceitual da aula

Roteiro visual

O conteúdo será organizado em três blocos. Primeiro, estudaremos autômatos com pilha e reconhecimento de linguagens livres de contexto. Depois, veremos gramáticas sensíveis ao contexto e linguagens mais expressivas. Por fim, discutiremos linguagens recursivas, máquinas de Turing e noções iniciais de decidibilidade.

Blocos da aula



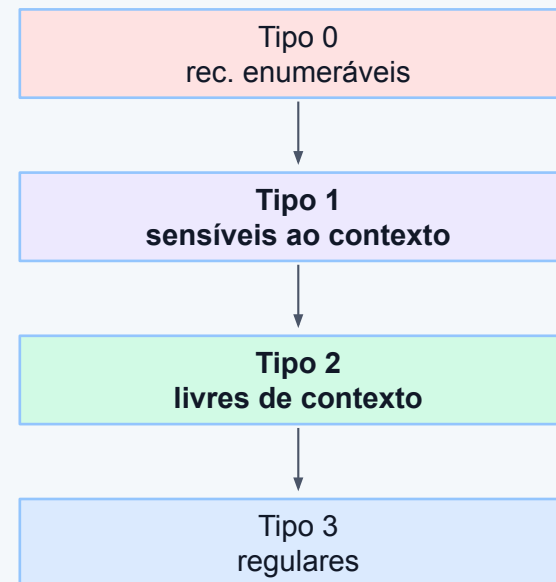
Hierarquia de Chomsky em perspectiva

Classes de linguagens e reconhecedores

A hierarquia de Chomsky organiza linguagens conforme o tipo de regra gramatical e o modelo reconhecedor associado. Linguagens regulares usam autômatos finitos; livres de contexto usam autômatos com pilha; sensíveis ao contexto usam memória linear limitada; e linguagens mais gerais se relacionam a máquinas de Turing.

[Link útil: Resumo da hierarquia de Chomsky](#)

Hierarquia de Chomsky



Por que autômatos finitos não bastam?

A necessidade de memória

Um autômato finito possui quantidade fixa de estados. Por isso, ele consegue lembrar apenas informações limitadas sobre a entrada. Linguagens como $a^n b^n$ exigem comparar quantidades arbitrárias de símbolos. Para isso, precisamos de uma memória que cresça com a entrada: a pilha.

Limite do autômato finito



$a^n b^n?$

precisa contar sem limite

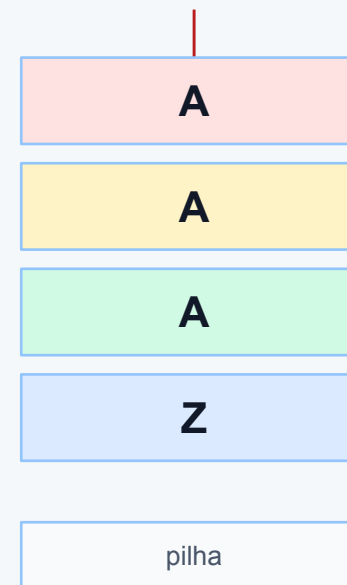
O que é uma pilha?

Memória LIFO

Pilha é uma estrutura de dados em que o último item inserido é o primeiro a ser retirado. Esse comportamento é chamado LIFO, do inglês last in, first out. Em linguagens formais, a pilha permite guardar marcas temporárias para conferir estruturas aninhadas ou contagens pareadas.

Estrutura de pilha

topo

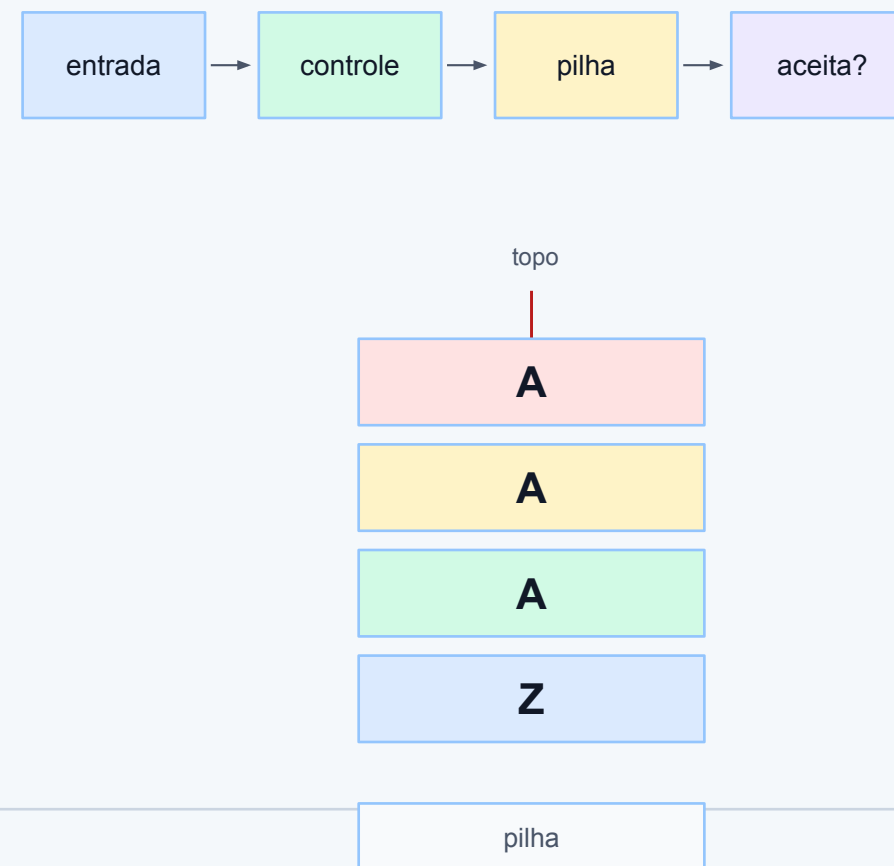


O que é um autômato com pilha?

Máquina abstrata com memória auxiliar

Um autômato com pilha, ou PDA, é uma máquina abstrata que lê a entrada símbolo por símbolo, muda de estado e manipula uma pilha. Diferentemente do autômato finito, ele pode empilhar, desempilhar ou manter símbolos, usando essa memória para reconhecer estruturas livres de contexto.

PDA = entrada + estados + pilha



Componentes formais de um PDA

Definição em alto nível

Um PDA é descrito por estados, alfabeto de entrada, alfabeto da pilha, função de transição, estado inicial, símbolo inicial da pilha e estados finais. Para uma primeira compreensão, o mais importante é saber que cada movimento depende do estado atual, da entrada e do topo da pilha.

Componentes

símbolo papel
Q estados
Σ entrada
Γ pilha
δ transições
F finais

Configuração instantânea

Fotografia de uma execução

Durante a execução, podemos representar a situação atual por três informações: estado corrente, parte da entrada ainda não lida e conteúdo da pilha. Essa configuração ajuda a simular o PDA passo a passo, mostrando exatamente por que uma palavra é aceita ou rejeitada.

Configuração instantânea

(q , entrada , pilha)

estado atual

símbolos restantes

conteúdo da pilha

próxima decisão

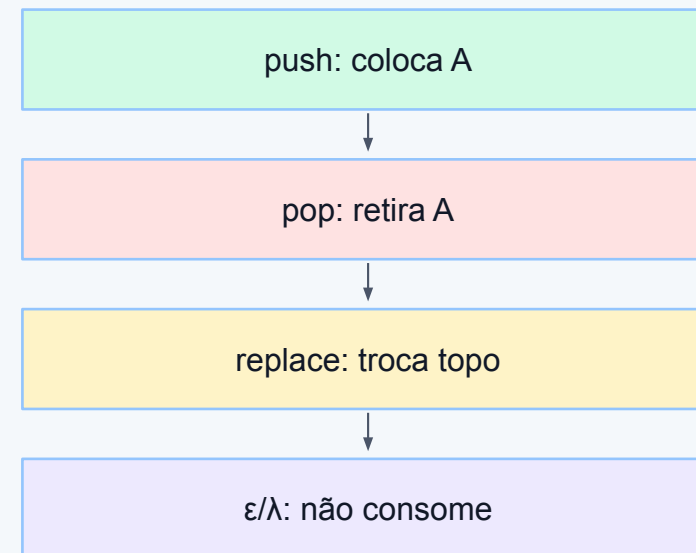
Operações da pilha

Empilhar, desempilhar e manter

A pilha permite três ações principais. Empilhar adiciona símbolos ao topo; desempilhar remove o símbolo do topo; substituir combina remoção e inserção. Em JFLAP, uma transição de PDA indica o símbolo lido, o símbolo esperado no topo da pilha e o que será colocado no lugar.

[Link útil: JFLAP](#)

Ações na pilha

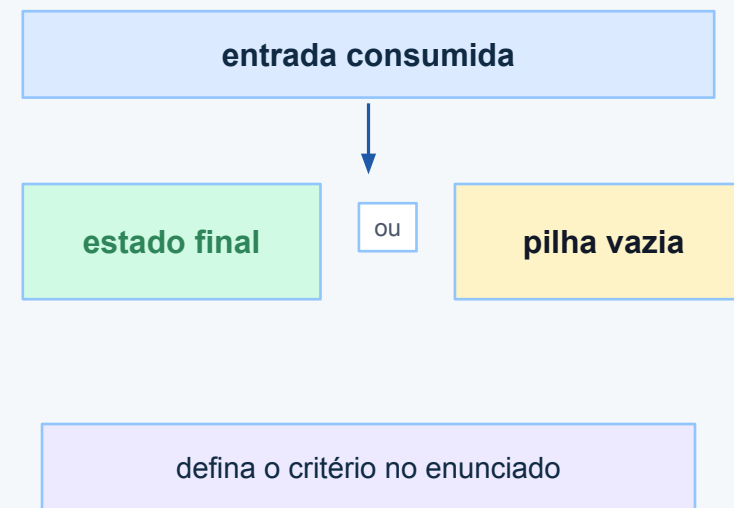


Critérios de aceitação

Estado final ou pilha vazia

Um PDA pode aceitar uma palavra de duas formas comuns: chegando a um estado final após consumir a entrada, ou deixando a pilha vazia ao final. Em atividades didáticas, é importante informar qual critério está sendo usado para evitar interpretações diferentes no mesmo exercício.

Como aceitar

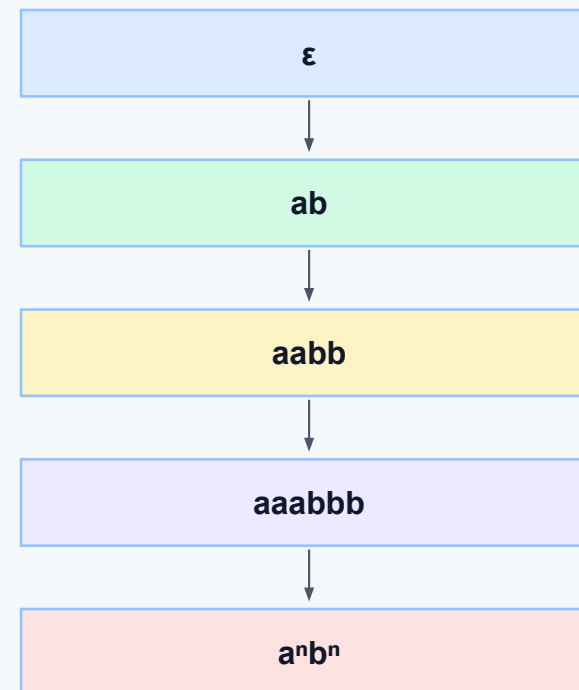


Linguagem exemplo: $a^n b^n$

Mesmo número de a e b

A linguagem $L = \{a^n b^n \mid n \geq 0\}$ contém ϵ , ab , $aabb$, $aaabbb$ e assim por diante. Ela não é regular porque exige comparar quantidades. Entretanto, é livre de contexto, pois uma pilha pode registrar cada a e retirar uma marca para cada b .

Contagem pareada

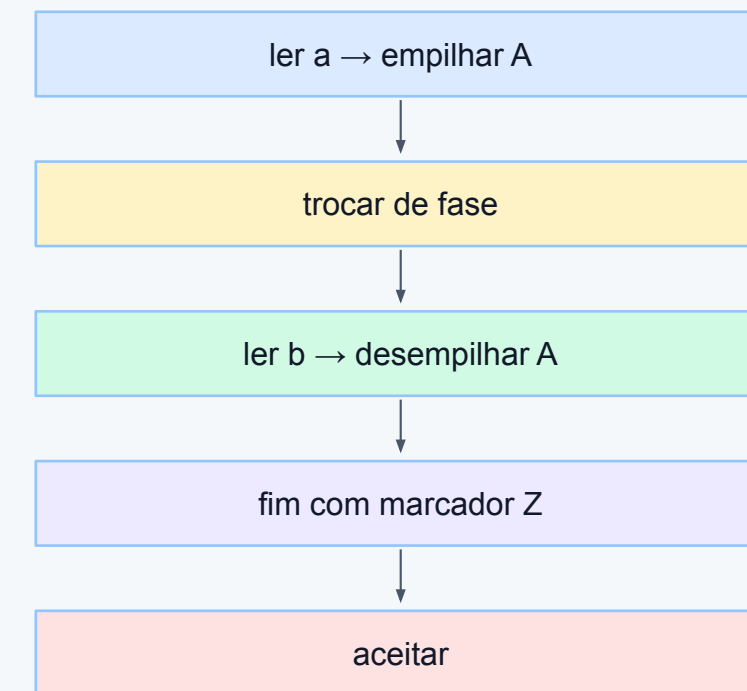


Estratégia do PDA para $a^n b^n$

Empilhar a, depois desempilhar b

A ideia do PDA é simples: enquanto lê símbolos a , empilha uma marca A para cada ocorrência. Quando começam os b , troca de fase e desempilha um A para cada b lido. Se a entrada acaba exatamente quando as marcas acabam, a palavra pertence à linguagem.

Duas fases



Simulação da palavra aabb

Execução passo a passo

Para a entrada aabb, o PDA lê o primeiro a e empilha A. Lê o segundo a e empilha outro A. Ao encontrar b, passa para a fase de desempilhar. Cada b remove uma marca. Como a pilha retorna ao marcador inicial no fim, a palavra é aceita.

Rastreamento de aabb

passo entrada pilha
0 aabb Z
1 abb AZ
2 bb AAZ
3 b AZ
4 ϵ Z ✓

Rejeição: quando a pilha denuncia erro

Entradas que não pertencem à linguagem

A palavra `aab` é rejeitada porque sobra uma marca `A` na pilha: foram lidos dois `a`, mas apenas um `b`. A palavra `abb` também é rejeitada porque aparece `b` demais, tentando retirar marca inexistente. Assim, a pilha funciona como memória de conferência.

Rejeições típicas

entrada motivo
<code>aab</code> sobra <code>A</code>
<code>abb</code> falta <code>A</code>
<code>ba</code> começa com <code>b</code>
<code>aba</code> ordem inválida

Determinístico × não determinístico

Escolha de caminhos

Um PDA determinístico possui no máximo uma ação possível para cada combinação relevante de estado, símbolo de entrada e topo da pilha. Já um PDA não determinístico pode ter escolhas. Em teoria, o não determinismo amplia a classe reconhecida em relação aos PDAs determinísticos.

Escolha de execução



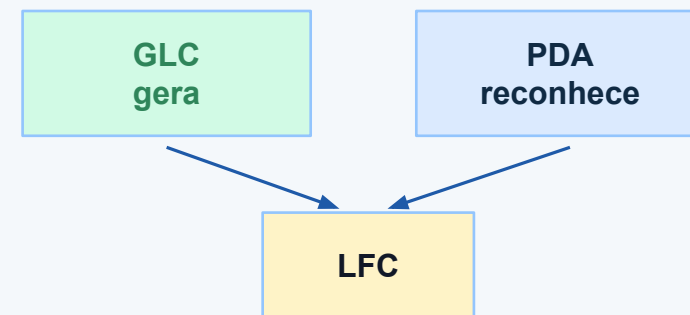
CFL determinísticas $\subset$ CFL

PDA e GLC: mesma classe de linguagens

Equivalência conceitual

As linguagens reconhecidas por autômatos com pilha não determinísticos são exatamente as linguagens livres de contexto. Isso conecta duas formas de pensar: a gramática gera palavras por regras, enquanto o PDA reconhece palavras usando estados, entrada e pilha.

Mesma classe

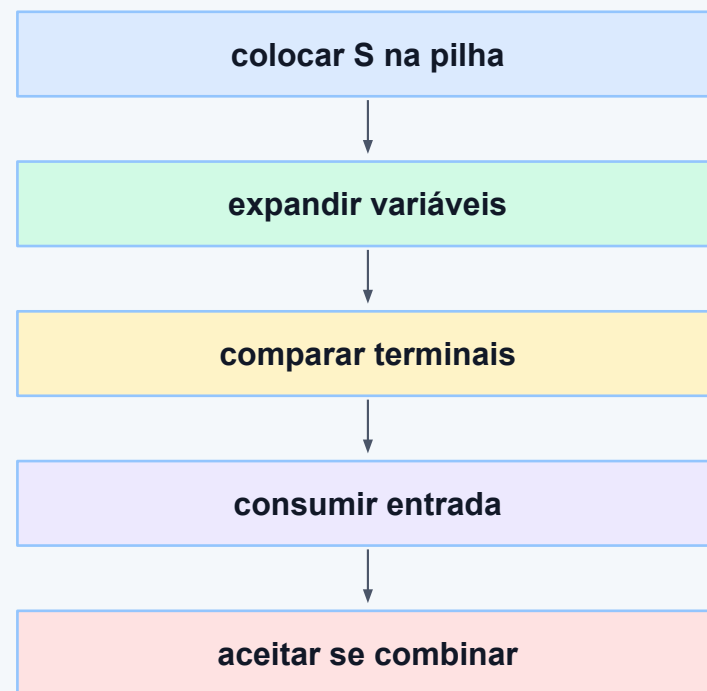


Conversão conceitual: GLC $\rightarrow$ PDA

Da geração ao reconhecimento

Uma forma de converter uma GLC em PDA é usar a pilha para guardar o que ainda precisa ser reconhecido. Variáveis na pilha são substituídas por produções da gramática; terminais na pilha são comparados com a entrada. Quando tudo combina, a palavra é aceita.

GLC $\rightarrow$ PDA

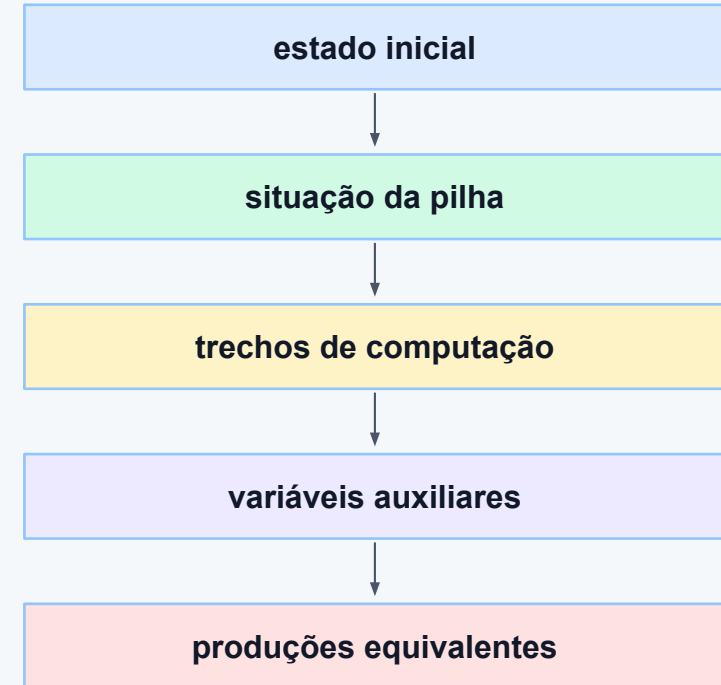


Conversão conceitual: PDA → GLC

Da máquina às regras

A conversão de PDA para GLC é mais trabalhosa, mas a ideia é criar variáveis que representam trechos de computação capazes de levar a pilha de uma situação a outra. Essa equivalência é importante porque confirma que gramáticas e máquinas descrevem a mesma classe.

PDA → GLC

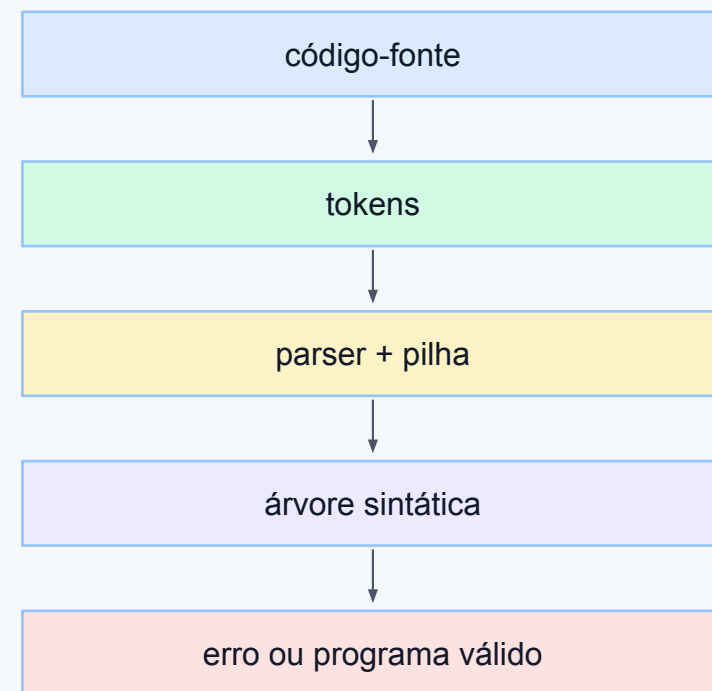


Aplicação em compiladores: parsing

Da entrada à árvore sintática

Em compiladores, autômatos com pilha ajudam a entender analisadores sintáticos. O parser recebe tokens produzidos pela análise léxica e verifica se a sequência respeita a gramática da linguagem. A pilha representa expectativas: símbolos que ainda precisam ser fechados ou reduzidos.

Pipeline de compilação



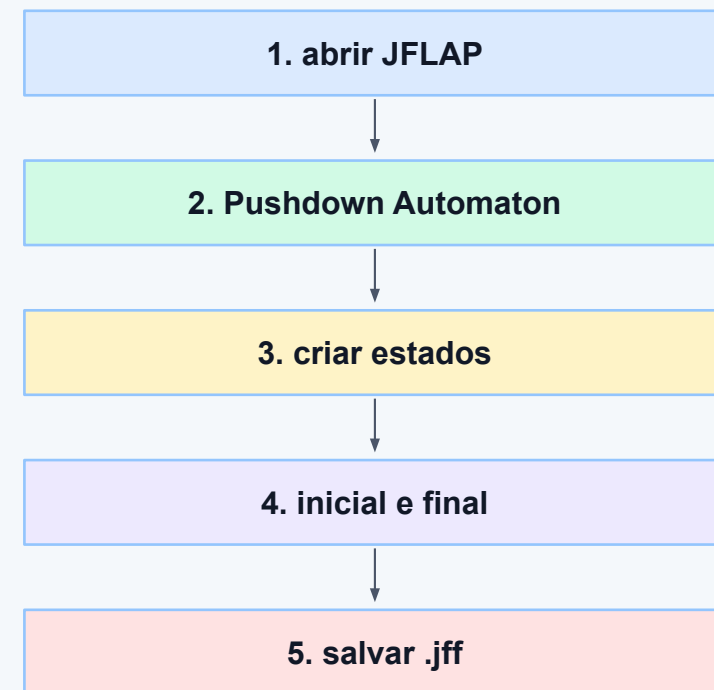
Prática no JFLAP: preparação

Ferramenta gratuita de apoio

Para praticar, use o JFLAP em laboratório. Baixe a ferramenta no site oficial, verifique se há Java disponível e abra a opção Pushdown Automaton. O objetivo da prática é construir um PDA para $a^n b^n$, simular entradas e comparar aceitação e rejeição.

[Link útil: JFLAP oficial](#)

Passos iniciais



Prática no JFLAP: transições

Formato input, pilha; novo conteúdo

No JFLAP, a transição de PDA registra três partes: símbolo lido da entrada, símbolo retirado do topo da pilha e sequência colocada de volta. Por exemplo, $a, Z; AZ$ significa ler a , remover Z e devolver A seguido de Z , empilhando uma marca.

Formato da transição

entrada, topo; novoTopo

rótulo efeito
$a, Z; AZ$ empilha A
$a, A; AA$ empilha A
$b, A; \lambda$ desempilha
$\lambda, Z; Z$ aceita/final

Prática no JFLAP: simular entradas

Multiple Run e validação

Após desenhar o PDA, use a simulação passo a passo ou múltipla. Teste palavras aceitas, como ϵ , ab, aabb e aaabbbb, e rejeitadas, como aab, abb e ba. A comparação mostra se as transições realmente representam a linguagem pretendida.

Testes no Multiple Run

entrada resultado
ϵ aceita
ab aceita
aabb aceita
aab rejeita
abb rejeita

Exercício guiado: parênteses balanceados

Outra linguagem livre de contexto

Considere a linguagem de parênteses corretamente balanceados, como ϵ , $()$, $(())$ e $()()$. Um PDA pode empilhar uma marca ao ler $($ e desempilhar ao ler $)$. Se tentar desempilhar sem marca ou terminar com marcas sobrando, a palavra deve ser rejeitada.

Parênteses balanceados



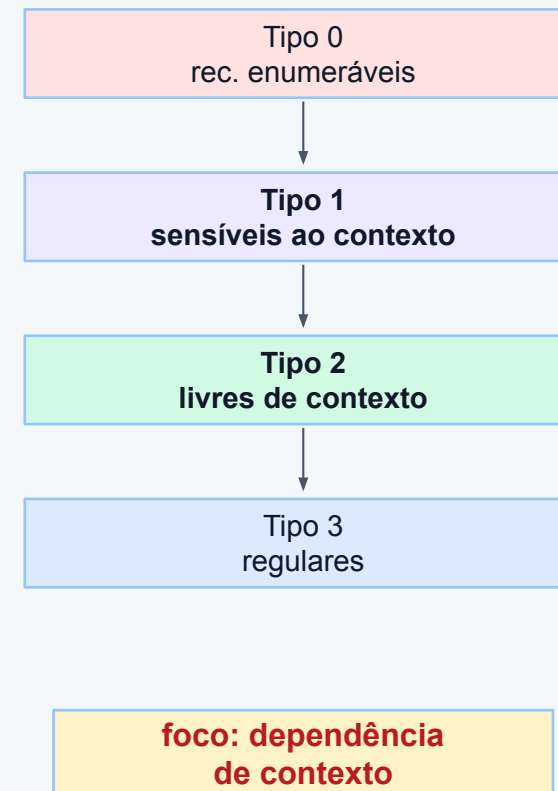
aceita: $()$ e $()()$

Gramáticas sensíveis ao contexto

Além das GLCs

Gramáticas sensíveis ao contexto pertencem ao Tipo 1 da hierarquia de Chomsky. Elas conseguem representar dependências mais fortes do que as livres de contexto. A ideia central é que a substituição pode depender dos símbolos vizinhos, isto é, do contexto em que ocorre.

Tipo 1



O que significa depender do contexto?

Regra influenciada pelos vizinhos

Em uma GLC, uma variável A pode ser substituída por α independentemente do que está ao redor. Em uma gramática sensível ao contexto, a produção pode considerar vizinhos, como $\alpha A \beta \rightarrow \alpha \gamma \beta$. Isso permite controlar relações mais complexas entre partes da palavra.

Regra com vizinhos

$$\alpha A \beta \rightarrow \alpha \gamma \beta$$

A muda considerando
o que está ao redor

Produções não contrativas

A palavra não diminui durante a derivação

Uma característica comum das gramáticas sensíveis ao contexto é que as produções não diminuem o tamanho da forma sentencial. Em termos simples, o lado direito da produção deve ter tamanho maior ou igual ao lado esquerdo, salvo exceções especiais para a palavra vazia.

Não contrair

$|\text{lado direito}| \geq |\text{lado esquerdo}|$

$AB \rightarrow ABC$

$A \rightarrow \epsilon$

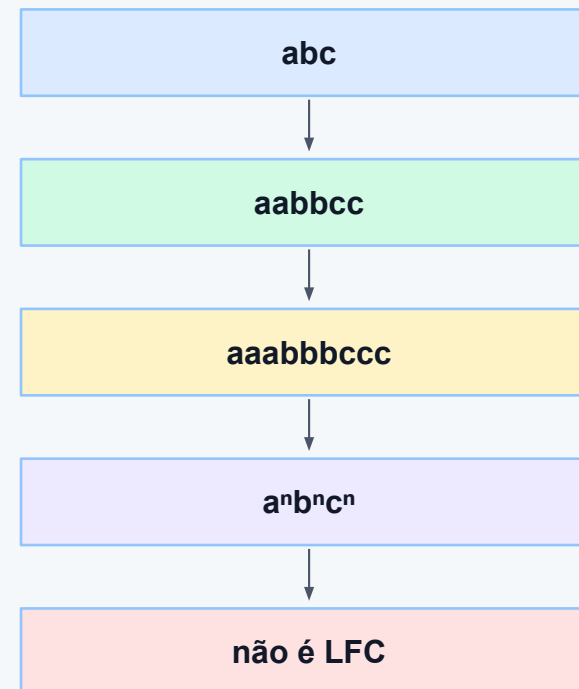
em geral, evitar diminuir

Linguagem exemplo: $a^n b^n c^n$

Três blocos com mesma quantidade

A linguagem $L = \{a^n b^n c^n \mid n \geq 1\}$ exige a mesma quantidade de a , b e c , sempre nessa ordem. Ela é um exemplo clássico de dependência que vai além das linguagens livres de contexto, pois relaciona três contagens simultaneamente.

Três contagens



Reconhecedor: autômato linearmente limitado

Memória limitada pelo tamanho da entrada

Linguagens sensíveis ao contexto são associadas a autômatos linearmente limitados. Esse modelo pode ser entendido como uma máquina de Turing cuja fita de trabalho é limitada por uma quantidade proporcional ao tamanho da entrada. Assim, há mais memória que uma pilha simples, mas não ilimitada.

Autômato linearmente limitado

entrada de tamanho n

fita limitada por $k \cdot n$

mais memória que pilha
menos que fita ilimitada

Comparando Tipo 2 e Tipo 1

Diferença de poder expressivo

Linguagens livres de contexto lidam muito bem com aninhamento e pareamento simples, como parênteses e $a^n b^n$. Linguagens sensíveis ao contexto conseguem expressar dependências mais rígidas, como $a^n b^n c^n$. A diferença aparece quando uma única pilha não é suficiente para controlar a estrutura.

Tipo 2 × Tipo 1

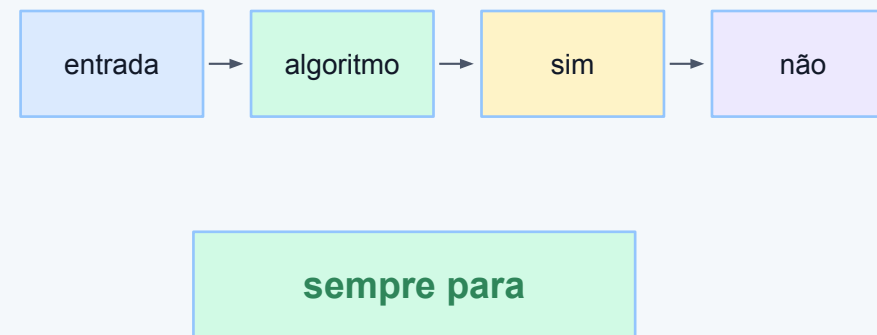
tipo exemplo
Livre contexto $a^n b^n$
Sensível contexto $a^n b^n c^n$
Pilha pareia dois blocos
LBA controla mais relações

Linguagens recursivas

Decisão com parada garantida

Uma linguagem é recursiva, ou decidível, quando existe um algoritmo que sempre para e responde corretamente se uma palavra pertence ou não a ela. A palavra “sempre” é essencial: para entradas aceitas e rejeitadas, o procedimento termina com uma resposta.

Decisor



Recursiva × reconhecível

Parar sempre ou apenas aceitar corretamente

Uma linguagem reconhecível, ou recursivamente enumerável, possui uma máquina que aceita as palavras pertencentes à linguagem. Porém, para palavras fora dela, a máquina pode rejeitar ou simplesmente executar para sempre. Linguagens recursivas exigem mais: a máquina deve parar em todos os casos.

Parada

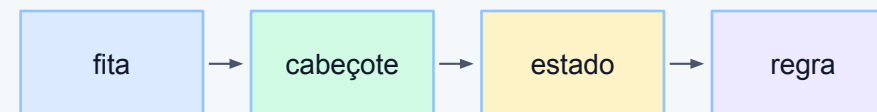
classe entrada fora
recursiva rejeita e para
rec. enum. pode não parar
diferença garantia de término

Máquinas de Turing como modelo

Computação abstrata geral

A máquina de Turing é um modelo teórico usado para estudar o que pode ser computado por algoritmo. Ela possui estados, fita, cabeçote de leitura/escrita e regras de transição. Embora seja abstrata, ajuda a discutir limites fundamentais da computação.

Máquina de Turing



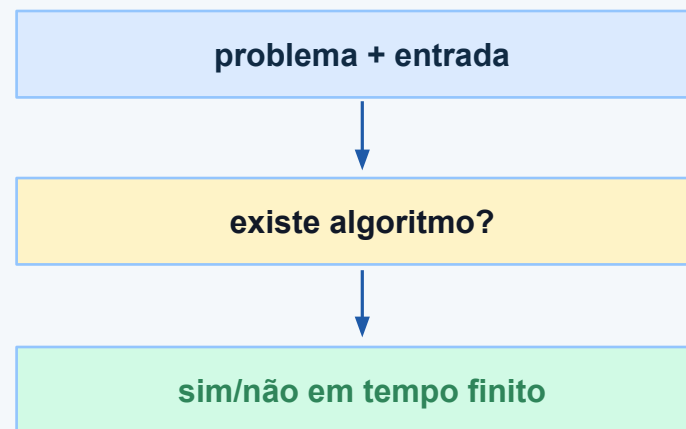
modelo geral de algoritmo

Decidibilidade: pergunta central

Existe algoritmo que sempre resolve?

Um problema é decidível quando existe um algoritmo capaz de responder corretamente, em tempo finito, para toda entrada possível. Em linguagens formais, perguntamos se há um procedimento geral para decidir pertinência, equivalência, vazio, ambiguidade ou outras propriedades.

Pergunta de decisão

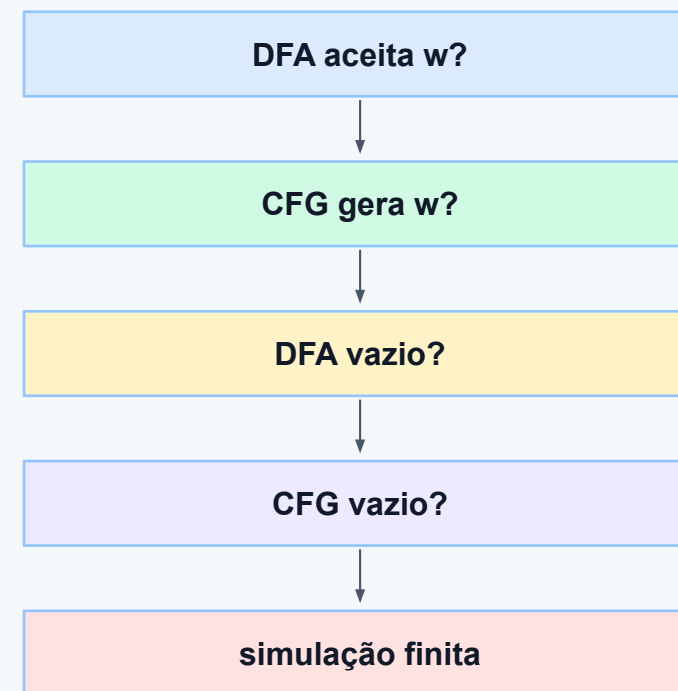


Problemas decidíveis típicos

Casos em que há algoritmo

Alguns problemas possuem algoritmos conhecidos. Por exemplo, verificar se uma palavra é aceita por um DFA é decidível, pois basta simular a leitura finita. Para GLCs, também é possível decidir se uma palavra pertence à linguagem usando algoritmos de parsing apropriados.

Exemplos decidíveis

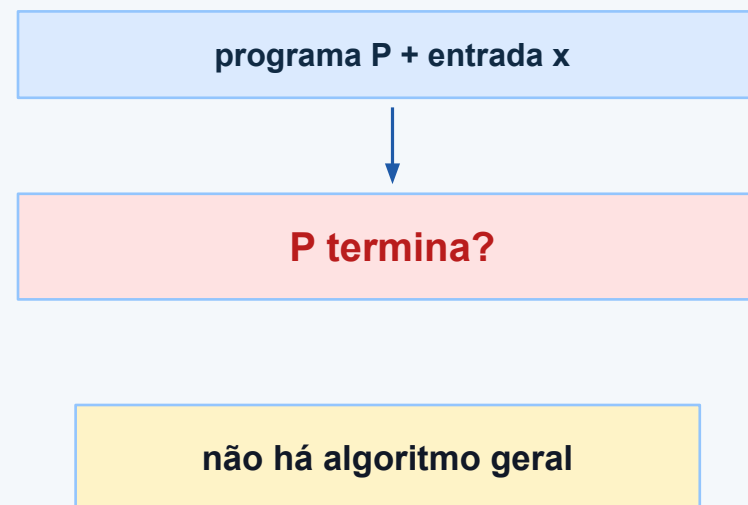


Problemas indecidíveis

Quando não existe solução geral

Um problema indecidível não possui algoritmo que resolva todos os casos possíveis com parada garantida. O exemplo clássico é o problema da parada: dado um programa e uma entrada, decidir se ele terminará ou executará para sempre. Esse limite afeta a teoria e a prática.

Problema da parada

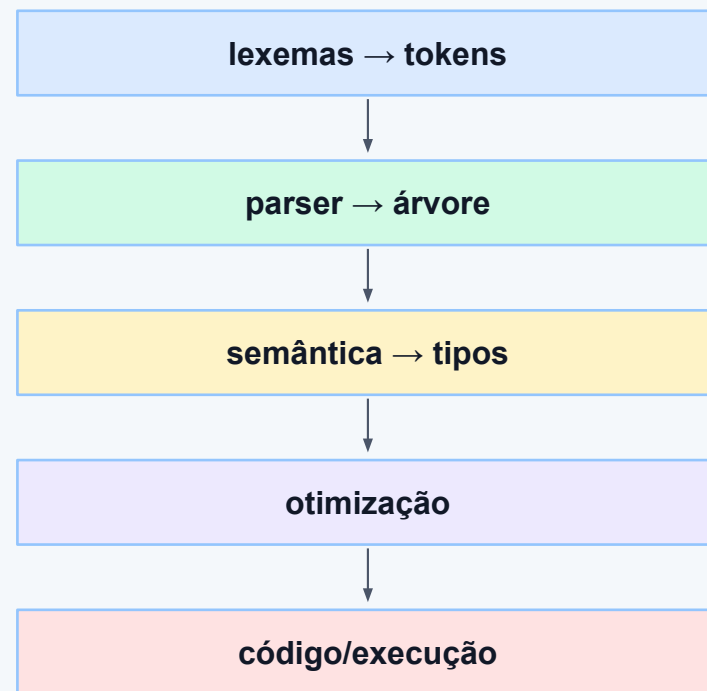


Consequências para compiladores

O que a teoria ensina à prática

Compiladores reais combinam várias etapas porque nenhuma técnica isolada resolve tudo de modo simples. Tokens podem ser tratados por autômatos finitos, sintaxe por gramáticas livres de contexto, e regras como tipos ou declaração antes de uso por análises semânticas posteriores.

Compilador em camadas

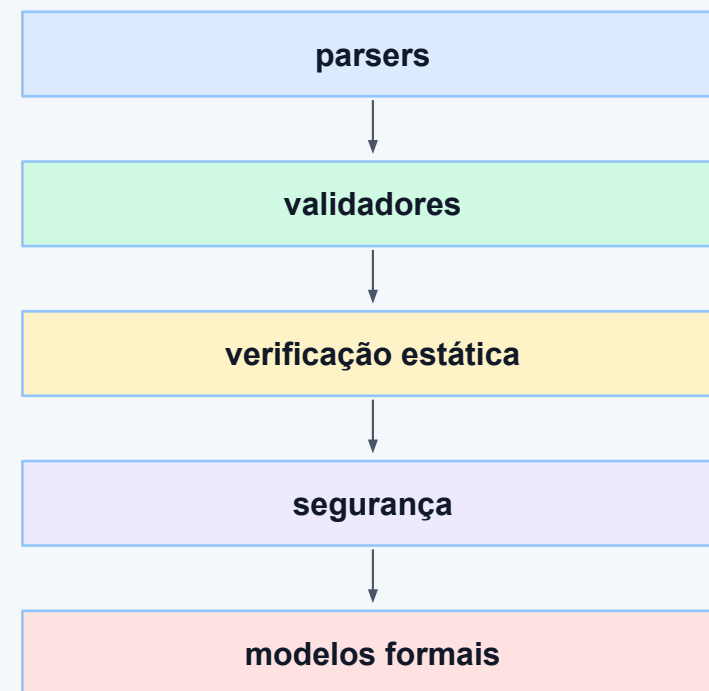


Noções contemporâneas

Verificação, segurança e ferramentas

Mesmo em tecnologias atuais, a teoria continua útil. Parsers, validadores, verificadores estáticos e ferramentas de segurança dependem de modelos formais para reconhecer estruturas, garantir propriedades e entender limites. Saber a classe da linguagem ajuda a escolher a técnica adequada.

Uso atual

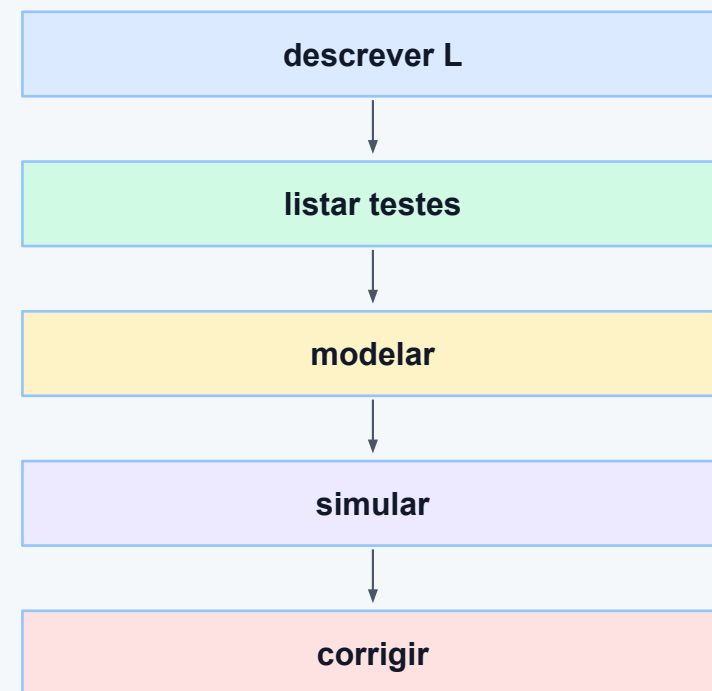


Roteiro prático sem custo

Como estudar com ferramentas gratuitas

Uma sequência prática eficiente é: escrever a linguagem em português, listar exemplos aceitos e rejeitados, propor uma gramática ou PDA, testar no JFLAP, registrar a simulação e explicar o critério de aceitação. Esse ciclo ajuda a transformar teoria em evidência verificável.

Ciclo de estudo

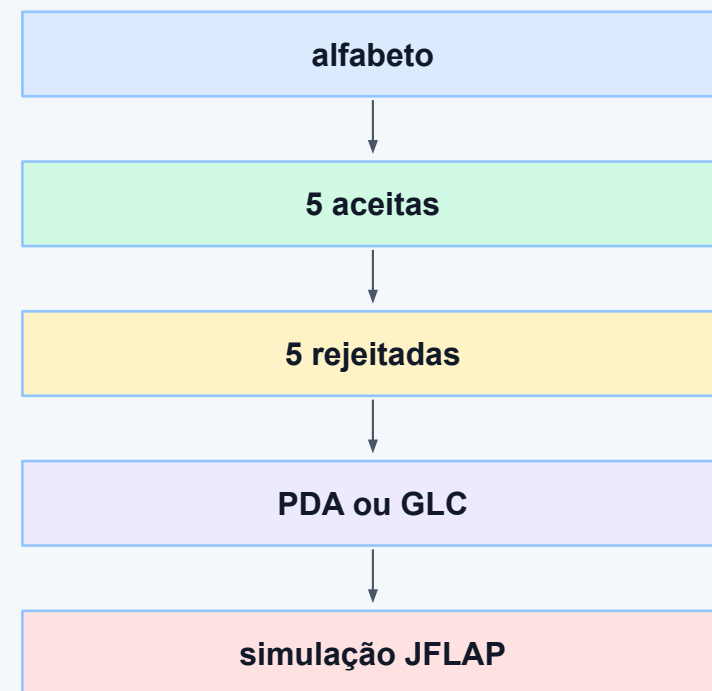


Atividade em grupo

Aplicação em sala

Cada grupo deve escolher uma linguagem simples, definir o alfabeto, escrever cinco palavras aceitas e cinco rejeitadas, propor um PDA ou gramática livre de contexto e simular pelo menos três casos no JFLAP. Ao final, o grupo deve justificar por que a pilha é necessária.

Entrega rápida

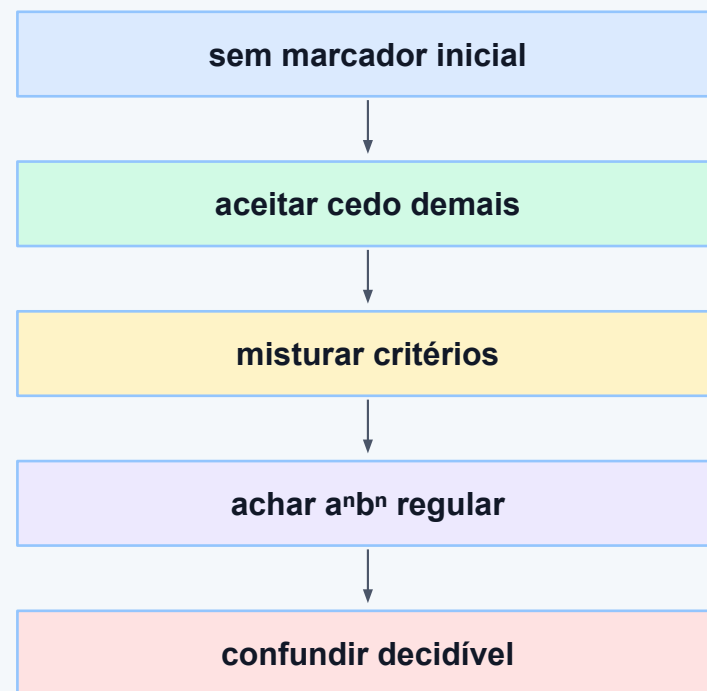


Erros comuns

O que evitar nos exercícios

Erros frequentes incluem esquecer o marcador inicial da pilha, aceitar antes de consumir toda a entrada, misturar critério de pilha vazia com estado final, tratar $a^n b^n$ como regular, remover escolhas necessárias em um NPDA e confundir reconhecível com decidível.

Atenção

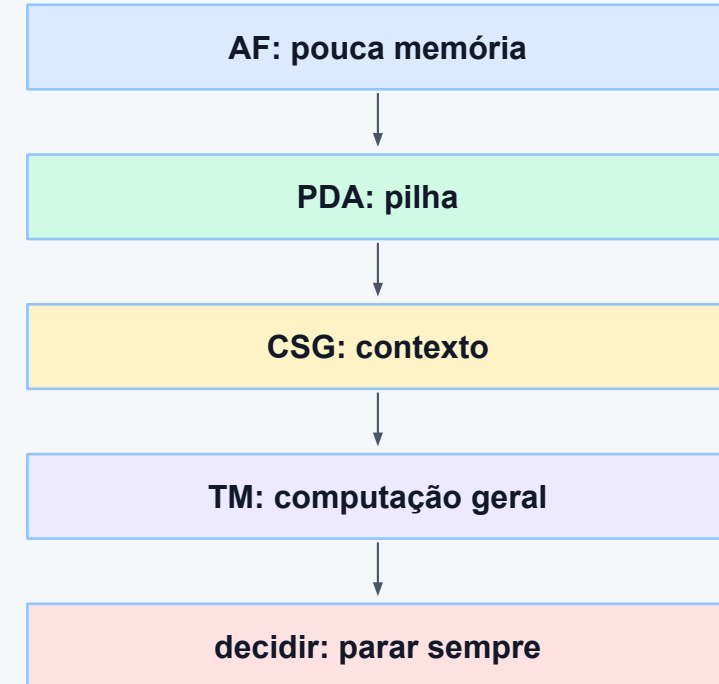


Síntese da aula

Ideias principais

Autômatos com pilha acrescentam memória aos autômatos finitos e reconhecem linguagens livres de contexto. Gramáticas sensíveis ao contexto expressam dependências ainda mais fortes. Linguagens recursivas exigem decisão com parada garantida, e decidibilidade mostra que alguns problemas não têm algoritmo geral.

Resumo em cadeia



Referências e materiais recomendados

Fontes para aprofundamento

Materiais da disciplina: aulas anteriores sobre hierarquia de Chomsky, linguagens regulares, autômatos finitos, conversões e gramáticas livres de contexto. Materiais complementares: JFLAP, OpenDSA, Stanford CS143/CS154, Hopcroft, Motwani e Ullman, Sipser e referências introdutórias sobre computabilidade e decidibilidade.

Materiais de apoio

